FactoryScheduler 維運指南（Maintainer Guide）

> 對象：負責安裝、打包、部署、排錯與擴充的工程人員。 版本：初版（engine + GUI 已完成，單機共享狀態）。

---

1. 系統概觀

把每天多來源工單（Legacy / APS）依機台能力與交期，自動排成各機台的作業時序； 支援抽件（Pull）優先、Machine_Calendar 停機避讓、管理員調整、操作員掃碼回報。 **目前只排製程 1（機台）**。

架構

```
app/
 ├── config.py            角色常數
 ├── settings.py          記住上次檔案路徑（JSON）
 ├── factory/             ← 核心引擎（無 GUI 相依）
 │    ├── normalize.py     文字/日期/時間/模組正規化
 │    ├── types.py         資料模型（MachineInfo / PackageTask / PullEvent ...）
 │    ├── masterdata.py    載入主檔 5 分頁
 │    ├── orders.py        訂單載入 + 四段轉譯 + 整包任務
 │    ├── deadlines.py     交期規則（最早開始 / 最晚完成 / SLA_REF）
 │    ├── pulls.py         抽件（單一筆、整包累計各等級量）
 │    ├── scheduler.py     時間軸貪婪排程（避停機 / 凍結）
 │    ├── output.py        介面格式 Excel 輸出
 │    ├── barcode.py       條碼轉換與比對
 │    ├── ledger.py        抽件匯入去重台帳
 │    ├── fservice.py      GUI 服務層（串接引擎 + 狀態庫）
 │    └── fstore.py        SQLite 狀態庫（狀態/優先/鎖定/快照）
 └── ui/
     ├── role_dialog.py           角色選擇
     ├── factory_admin_window.py  管理員：每機台時序
     └── factory_operator_window.py 操作員：掃描回報
```

進入點： - `run_factory.py` — CLI 全流程（批次 / 除錯，無 GUI） - `run_factory_gui.py` — GUI， `--role admin|operator` 或不帶參數跳選單 - `main_factory_admin.py` / `main_factory_operator.py` — 給打包成固定角色 exe 用

---

2. 環境與安裝

需求：Python 3.12、`requirements.txt`（PySide6、pandas、openpyxl、xlrd；打包用 pyinstaller）。

封閉網路安裝

1. 連網機器下載 wheels：`bash pip download -r requirements.txt -d wheels`
2. 封閉機器離線安裝：`bash pip install --no-index --find-links wheels -r requirements.txt`

Linux 開發（本機 venv）

```
cd /HDD/tools/git/thales
source .venv/bin/activate      # 或直接 .venv/bin/python
```

> Linux 跑 GUI 需 X11，且 Qt6 需 `libxcb-cursor0`（`sudo apt install libxcb-cursor0`），啟動時建議 `QT_QPA_PLATFORM=xcb`。正式交付是 Windows exe，無此問題。

---

3. 打包成 Windows exe

執行 `build_exe_factory.bat`，`dist\` 產出三支：- `FactoryScheduler.exe` — 啟動選角色 - `FactoryScheduler_管理員.exe` / `FactoryScheduler_操作員.exe` — 固定角色

部署方式三擇一：- A) 只發單一檔，啟動時選角色 - B) 分發兩支固定角色 exe - C) 單一檔 + 兩個捷徑分別加 `--role admin` / `--role operator`

---

4. 部署與資料

單機共享

管理員與操作員共用「同一資料夾下」的 `factory_state.db`（SQLite）。管理員排程 → 寫入快照與狀態 → 操作員讀快照掃描/回報 → 管理員每 5 秒自動反映。

資料檔

類型	位置	格式
主檔	`FactoryScheduler/Schema/`	1 個 xlsx，5 分頁：Machine_Info / Machine_Calendar / Customer_Deadline / Product_Master / Product_Setup

類型	位置	格式
日常訂單	`FactoryScheduler/Orders/`	Legacy `.xls` / `.txt` (以 `|` 分隔)、APS `.xlsx`
抽件	`FactoryScheduler/Pulls/`	`.txt` / `.xls`，欄位含 `ID, CUSTOMER_NAME, FILE_NAME, WORKORDER_ID, PACKAGE_ID, PULL_TYPE, PULL_TIME, STATUS`
去重台帳	`FactoryScheduler/Logs/pull_imported.json`	以 `ID+FILE_NAME` 記錄已匯入抽件
輸出	`FactoryScheduler/Output/`	介面格式 xlsx（每機台一分頁 + 摘要）

> `FactoryScheduler.ini` 為規格參考（資料夾監控 / 觸發時間），**初版尚未接入**；目前路徑由 GUI 檔案對話框或 CLI 參數指定。

---

5. 狀態重置（常見）

```
pkill -f run_factory_gui                                    # 1) 先關所有 GUI
rm -f factory_state.db FactoryScheduler/Logs/pull_imported.json   # 2) 砍狀態 + 抽件台帳
```

- **務必先關視窗再砍**：執行中的行程會把檔案寫回去（這是「砍了還有狀態」的主因）。
- 只砍 `factory_state.db` 而留著台帳 → 重載同一份抽件會「重複略過、套用 0」。

---

6. 排程邏輯摘要

1. **四段轉譯**：客戶別名 → 產品代號 → 替代版面(Product_Master) → 群組/必備模組(Product_Setup) → 合格機台（模組涵蓋 + 群組 UPH>0）。
2. **交期**：最早開始 = 到檔日 + 客戶 Start Time；最晚完成 = 承諾日 +（SLA_REF 命中時間，否則 SLA Std，否則 23:59）。SLA_REF 代碼需完全相等（`D01` ≠ `D00`）。
3. **抽件**：每筆 = 1 張，累計到整包對應等級量（EMER_1/2/Std），整包標優先、交期收緊、加 Pull_Setup_Min；介面整包一列呈現。
4. **排程**：時間前推貪婪；排序＝抽件/優先 → 交期(EDD) → 量大先；每任務挑「完工最早」的合格機台，避開 Machine_Calendar 停機；狀態為處理中/已完成或鎖定者凍結在原機台。
5. **加工時間** = Σ(行數量 ÷ 機台該群組 UPH) × 60 分。

---

7. 排錯

症狀	原因 / 解法
`numpy >= 2.0.2` / `C extension: None not built`	用到系統 python，非 venv。請 `source .venv/bin/activate` 後再跑。
Linux GUI 起不來、`xcb` 無法載入	缺 `libxcb-cursor0` → `apt install libxcb-cursor0`；啟動加 `QT_QPA_PLATFORM=xcb`。
`.xls` 讀不到	缺 `xlrd` → 依 requirements 安裝。
抽件「套用 0、重複略過 N」	已在去重台帳；要重跑請刪 `pull_imported.json`。
砍了 DB 還有狀態	GUI 還開著把檔寫回。先關視窗再砍。
操作員掃不到工單	管理員尚未載入並排程（操作員讀的是排程快照）。

---

8. 初版限制 / 後續

- 只排製程 1；Insert / Manual 階段尚未納入。
- 管理員「重新排程」為手動按鈕（操作員狀態變更只刷新顯示，不自動重排）。
- `FactoryScheduler.ini` 的資料夾監控 / 定時觸發尚未接入（目前手動選檔）。
- 時間軸排程用貪婪演算法；如需最佳化可改 CP-SAT（OR-Tools）。
- 待確認資料規則：`D01` 是否應補進主檔 SLA_REF 清單。